

EXPERIMENT- 1

1. Experiment Title: Demonstrate inline, internal, and external JavaScript; use console methods; create a webpage showing user information and a welcome message.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

External CSS.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Experiment 1</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div>
    <h1>Welcome to the Lab!</h1>
    <p><strong>Name:</strong> Ashlesha Singh</p>
    <p><strong>PRN:</strong> 24070521221</p>
    <button onclick="alert('External CSS active!')">Test External CSS</button>
  </div>
</body>
</html>
```

style.css (External CSS)

```
/* External CSS used to style the page */
body {
  font-family: 'Segoe UI', sans-serif;
  background-color: #f4f4f9;
  display: flex;
  justify-content: center;
  padding-top: 50px;
}

div {
  background-color: white;
  padding: 40px;
  border-radius: 8px;
  border-top: 5px solid blue; /* Adds a nice accent */
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}
```

```

    text-align: center;
    width: 350px;
}

h1 {
    color: #333;
    font-size: 22px;
}

button {
    padding: 10px 20px;
    background-color: blue;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
}

button:hover {
    background-color: darkblue;
}

```

Inline CSS.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Experiment 1</title>
</head>
<body style="font-family: 'Segoe UI', sans-serif; background-color: #f4f4f9; display:
flex; justify-content: center; padding-top: 50px; margin: 0;">

    <div style="background-color: white; padding: 40px; border-radius: 8px; border-top:
5px solid blue; box-shadow: 0 2px 10px rgba(0,0,0,0.1); text-align: center; width:
350px;">

        <h1 style="color: #333; font-size: 22px;">Welcome to the Lab!</h1>

        <p><strong>Name:</strong> Ashlesha Singh</p>
        <p><strong>PRN:</strong> 24070521221</p>

        <button onclick="alert('Inline CSS active!')"
            style="padding: 10px 20px; background-color: blue; color: white; border: none;
border-radius: 4px; cursor: pointer; font-size: 14px;">

```

```
        Test Inline CSS
    </button>
</div>
</body>
</html>
```

Internal CSS.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Experiment 1</title>

    <style>
        body {
            font-family: 'Segoe UI', sans-serif;
            background-color: #f4f4f9;
            display: flex;
            justify-content: center;
            padding-top: 50px;
            margin: 0;
        }

        div {
            background-color: white;
            padding: 40px;
            border-radius: 8px;
            border-top: 5px solid blue;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
            text-align: center;
            width: 350px;
        }

        h1 {
            color: #333;
            font-size: 22px;
        }

        button {
            padding: 10px 20px;
            background-color: blue;
            color: white;
            border: none;
            border-radius: 4px;
```

```

        cursor: pointer;
        font-size: 14px;
    }

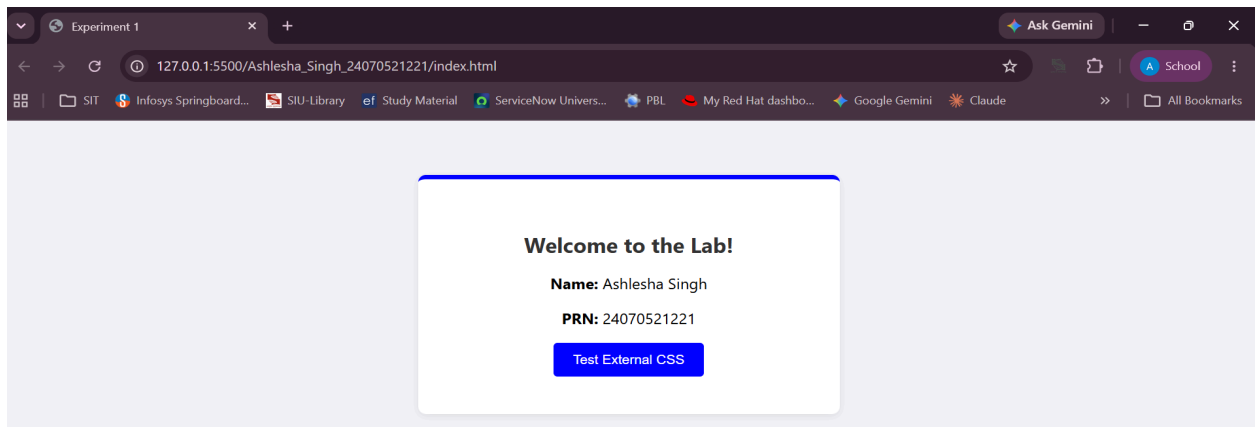
    button:hover {
        background-color: darkblue;
    }
</style>
</head>
<body>

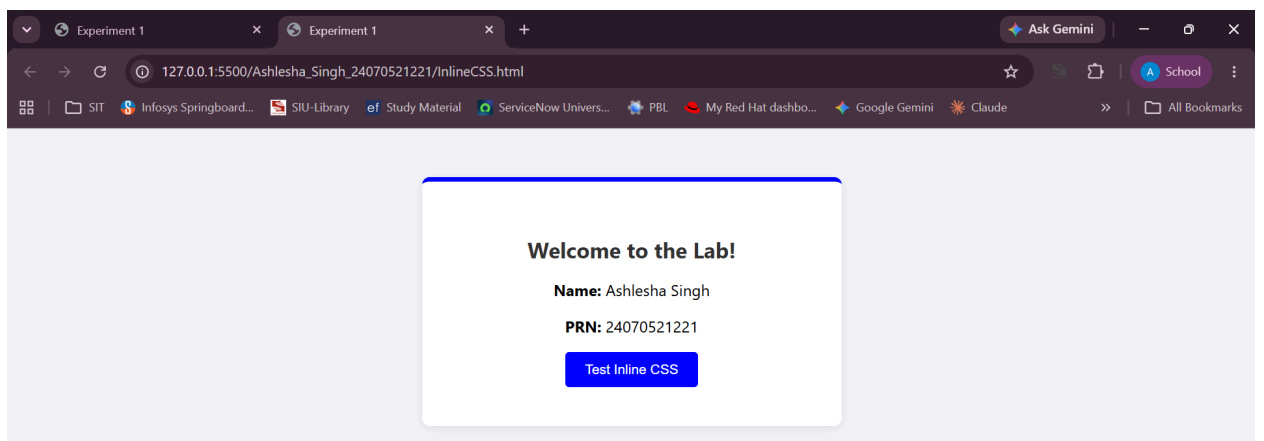
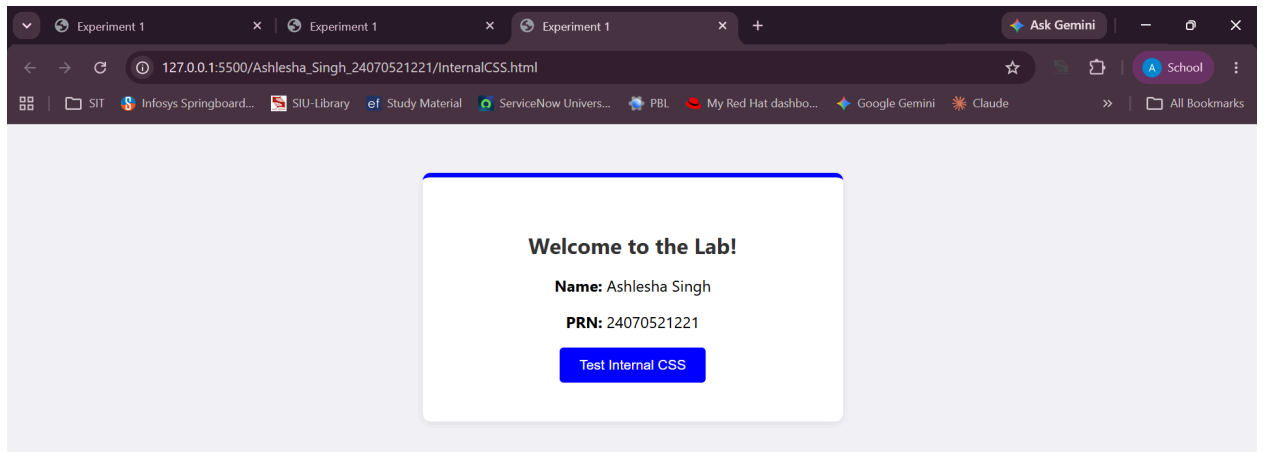
<div>
    <h1>Welcome to the Lab!</h1>
    <p><strong>Name:</strong> Ashlesha Singh</p>
    <p><strong>PRN:</strong> 24070521221</p>
    <button onclick="alert('Internal CSS active!')">Test Internal CSS</button>
</div>

</body>
</html>

```

4.Output:





5. Result/Conclusion: The experiment was successfully performed by implementing JavaScript through three different integration methods. Console functions were verified using browser developer tools, and the required environment information was displayed on the webpage.

CASE STUDY - 1

1. Case Study Title: Create a webpage for the college department and link the student information page with it and write a Java Script Program to Display “ Welcome to SIT Nagpur ” in an alert Box.

2. Software/Tools Required: VS Code , Google Chrome

3. Case Study Program Code:

CS1_Main.html

```
<html>

<head>

<title>SIT Nagpur - Department Page</title>

<style>

    body {

        font-family: Arial, sans-serif;

        margin: 40px;

        background-color: #f4f4f9;

        color: #333;

    }

    .container {

        max-width: 600px;

        background: #fff;

        padding: 25px;

        border-radius: 8px;

        box-shadow: 0 2px 5px rgba(0,0,0,0.1);

    }
```

```
h1 {
    color: #1a365d;
}

.btn {
    display: inline-block;
    margin-top: 15px;
    padding: 10px 15px;
    background-color: #3182ce;
    color: #fff;
    text-decoration: none;
    border-radius: 5px;
}

.btn:hover {
    background-color: #2b6cb0;
}

</style>

</head>

<body>

    <div class="container">

        <h1>Department of Computer Science & Engineering</h1>

        <p>Welcome to the official department portal of SIT Nagpur. We offer
state-of-the-art facilities and top-tier academic guidance to shape the innovators of
tomorrow.</p>

        <!-- Linking the student information page -->
```

```
<a href="CS1_Stud.html" class="btn">View Student Information</a>
</div>
```

```
<!-- JavaScript Program to display the Alert Box -->
```

```
<script>
    window.onload = function() {
        alert("Welcome to SIT Nagpur");
    };
</script>
```

```
</body>
```

```
</html>
```

CS1_Stud.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Information</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
            background-color: #f4f4f9;
            color: #333;
        }
        .container {
```



```
    max-width: 600px;
    background: #fff;
    padding: 25px;
    border-radius: 8px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

h2 {
    color: #1a365d;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
}

th, td {
    padding: 10px;
    border: 1px solid #ddd;
    text-align: left;
}

th {
    background-color: #edf2f7;
}

.back-link {
    display: inline-block;
    margin-top: 20px;
```



```

        <td>3rd Year</td>

    </tr>

</tbody>

</table>

<!-- Link back to Department page -->

    <a href="CS1_Main.html" class="back-link">← Back to Department
Page</a>

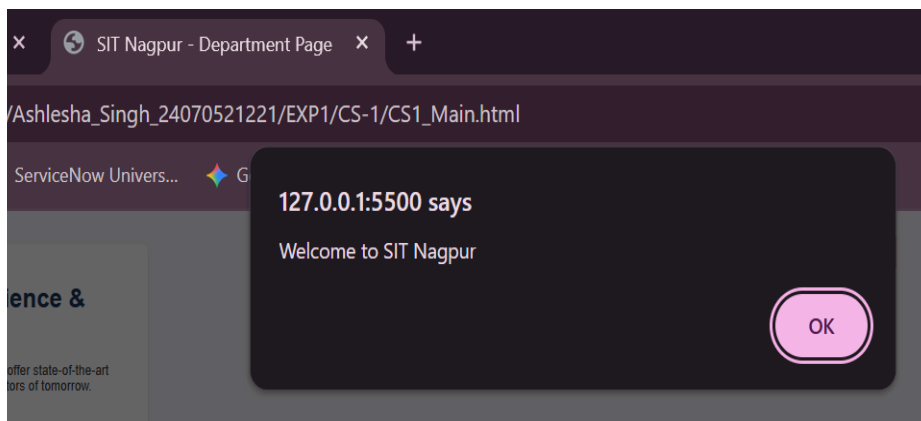
</div>

</body>

</html>

```

4. Output:



Department of Computer Science & Engineering

Welcome to the official department portal of SIT Nagpur. We offer state-of-the-art facilities and top-tier academic guidance to shape the innovators of tomorrow.

[View Student Information](#)

Student Information Page

Roll No	Student Name	Year
24070521221	Ashlesha Singh	3rd Year

[← Back to Department Page](#)

5. Result/Conclusion: In conclusion, this case study successfully demonstrates the design and development of a functional college department webpage integrated with a linked student information portal. By leveraging HTML for structure, CSS for presentation, and JavaScript for dynamic interactivity, the project effectively fulfills its technical objectives. The successful execution of the JavaScript alert box displaying "Welcome to SIT Nagpur" ensures an interactive and engaging user experience upon accessing the site. Ultimately, this implementation provides a streamlined, accessible, and professional web interface that serves as a practical blueprint for institutional information management.

EXPERIMENT- 2

1. Experiment Title: Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

```
<!DOCTYPE html>

<html>

<head>

    <title>Billing Calculator</title>

</head>

<body>


    <h2>Billing Calculator</h2>


    <script>


const GST = 0.18;


var customerName = prompt("Enter Customer Name: ");
var productName = prompt("Enter Product Name: ");


var quantity = Number(prompt("Enter quantity"));
let price = Number(prompt("Enter price per Item"));
```

```
let total = quantity * price;  
let gstAmount = total * GST;  
let finalBill = total + gstAmount;
```

```
const bill = {  
  customer: customerName,  
  product: productName,  
  quantity,  
  price,  
  total,  
  gstAmount,  
  finalBill  
};
```

```
const {  
  customer,  
  product,  
  gstAmount: gst,  
  finalBill: amount  
} = bill;
```

```
document.write(`<h3>Billing Receipt</h3>`);  
document.write(`<p>Customer: ${customer}</p>`);  
document.write(`<p>Product: ${product}</p>`);  
document.write(`<p>Quantity: ${quantity}</p>`);
```

```
document.write(`<p>Price per Item: ₹${price}</p>`);  
document.write(`<p>Total: ₹${total}</p>`);  
document.write(`<p>GST (18%): ₹${gst.toFixed(2)}</p>`);  
document.write(`<h3>Final Bill: ₹${amount.toFixed(2)}</h3>`);  
  
</script>  
</body>  
</html>
```

4.Output:

127.0.0.1:5500 says

Enter Customer Name:

OK Cancel

127.0.0.1:5500 says

Enter Product Name:

OK Cancel

127.0.0.1:5500 says

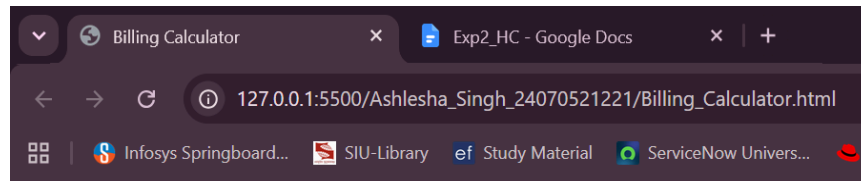
Enter quantity

OK Cancel

127.0.0.1:5500 says

Enter price per Item

OK Cancel



Billing Calculator

Billing Receipt

Customer: Ashlesha Singh - 24070521221

Product: Ashlesha Dell Laptop-221

Quantity: 221

Price per Item: ₹221

Total: ₹48841

GST (18%): ₹8791.38

Final Bill: ₹57632.38

5.Result/Conclusion:The HTML and JavaScript-based billing calculator was successfully implemented and executed. The program successfully accepts the customer name, product name, quantity, and price via user prompts, calculates the subtotal, applies an 18% GST, and dynamically renders a formatted billing receipt directly onto the web page.

CASE STUDY - 2

1. Case Study Title: Develop the webpage for an online shopping website using HTML,CSS and HTML5 tags And calculate the bill.

- Customer Name
- Mobile Number
- Invoice Number
- Product Name
- Quantity (in Kg)
- Rate per Kg (₹)
- Discount (%)
- GST (%)
- Packing Charges (₹)
- Payment Mode (Cash/Card/UPI)
- Membership (Yes/No)

2. Software/Tools Required: VS Code , Google Chrome

3. Case Study Program Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Online Shopping Billing Calculator</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      margin: 20px;

      background-color: #f4f4f9;
```

```
}  
  
.container {  
    max-width: 500px;  
    background: #ffffff;  
    padding: 20px;  
    border-radius: 8px;  
    box-shadow: 0px 0px 10px rgba(0, 0, 0, 0.1);  
}  
  
h2, h3 {  
    color: #333;  
}  
  
p {  
    margin: 6px 0;  
    font-size: 15px;  
}  
  
hr {  
    border: 0;  
    height: 1px;  
    background: #ccc;  
    margin: 15px 0;  
}  
  
.final-amount {  
    color: #2e7d32;  
    font-size: 18px;  
    font-weight: bold;
```

```
    }  
  </style>  
</head>  
<body>  
  
  <div class="container">  
  
    <h2>Online Shopping Billing Calculator</h2>  
  
    <div id="receipt"></div>  
  
  </div>  
  
  <script>  
  
    // 1. Collecting Inputs as specified in the Case Study  
  
    var customerName = prompt("Enter Customer Name:");  
    var mobileNumber = prompt("Enter Mobile Number:");  
    var invoiceNumber = prompt("Enter Invoice Number:");  
    var productName = prompt("Enter Product Name:");  
  
    var quantity = Number(prompt("Enter Quantity (in Kg):"));  
    var ratePerKg = Number(prompt("Enter Rate per Kg (₹):"));  
    var discountPercent = Number(prompt("Enter Discount (%):"));  
    var gstPercent = Number(prompt("Enter GST (%):"));  
    var packingCharges = Number(prompt("Enter Packing Charges (₹):"));  
  
    var paymentMode = prompt("Enter Payment Mode (Cash/Card/UPI):");  
    var isMember = prompt("Membership Status (Yes/No):");
```

```
// 2. Calculations
```

```
let baseAmount = quantity * ratePerKg;
```

```
let discountAmount = (baseAmount * discountPercent) / 100;
```

```
let amountAfterDiscount = baseAmount - discountAmount;
```

```
let gstAmount = (amountAfterDiscount * gstPercent) / 100;
```

```
let finalBill = amountAfterDiscount + gstAmount + packingCharges;
```

```
// 3. Storing in an Object
```

```
const bill = {
```

```
  customerName,
```

```
  mobileNumber,
```

```
  invoiceNumber,
```

```
  productName,
```

```
  quantity,
```

```
  ratePerKg,
```

```
  discountPercent,
```

```
  discountAmount,
```

```
  gstPercent,
```

```
  gstAmount,
```

```
  packingCharges,
```

```
  paymentMode,
```

```
  isMember,
```

```
  finalBill
```

```
};
```

```
// 4. Destructuring Object for Output
```

```
const {  
    customerName: cName,  
    mobileNumber: mNo,  
    invoiceNumber: invNo,  
    productName: pName,  
    quantity: qty,  
    ratePerKg: rate,  
    discountAmount: disc,  
    gstAmount: gst,  
    packingCharges: packing,  
    paymentMode: mode,  
    isMember: member,  
    finalBill: total  
} = bill;
```

```
// 5. Displaying Results on Page
```

```
document.getElementById("receipt").innerHTML = `  
    <h3>Invoice Receipt</h3>  
    <p><strong>Invoice No:</strong> ${invNo}</p>  
    <p><strong>Customer Name:</strong> ${cName}</p>  
    <p><strong>Mobile No:</strong> ${mNo}</p>  
    <p><strong>Membership Status:</strong> ${member}</p>
```

```

<hr>

<p><strong>Product Name:</strong> ${pName}</p>

<p><strong>Quantity:</strong> ${qty} Kg</p>

<p><strong>Rate per Kg:</strong> ₹${rate.toFixed(2)}</p>

<p><strong>Base Amount:</strong> ₹${baseAmount.toFixed(2)}</p>

<p><strong>Discount    (${discountPercent}%):</strong>
-₹${disc.toFixed(2)}</p>

<p><strong>GST (${gstPercent}%):</strong> +₹${gst.toFixed(2)}</p>

<p><strong>Packing Charges:</strong> +₹${packing.toFixed(2)}</p>

<hr>

<p><strong>Payment Mode:</strong> ${mode}</p>

<p class="final-amount">Final Bill Amount: ₹${total.toFixed(2)}</p>

`;

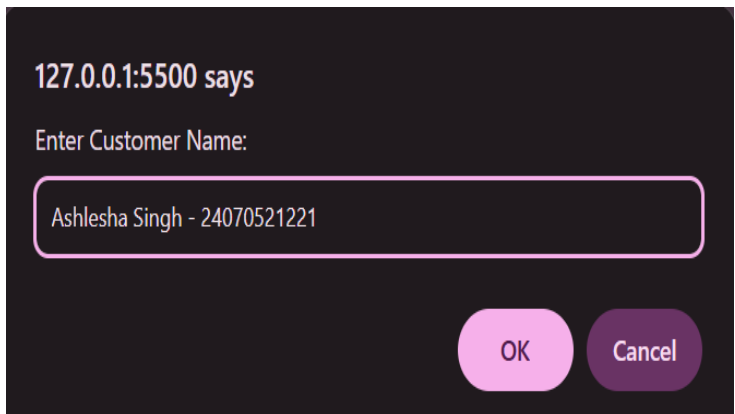
</script>

</body>

</html>

```

4. Output:



127.0.0.1:5500 says

Enter Customer Name:

Ashlesha Singh - 24070521221

OK Cancel

127.0.0.1:5500 says

Enter Mobile Number:

1231231231|

OK

Cancel

127.0.0.1:5500 says

Enter Invoice Number:

101

OK

Cancel

127.0.0.1:5500 says

Enter Product Name:

Laptop Asus

OK

Cancel

127.0.0.1:5500 says

Enter Quantity (in Kg):

21

OK

Cancel

127.0.0.1:5500 says

Enter Rate per Kg (₹):

50

OK

Cancel

127.0.0.1:5500 says

Enter Discount (%):

15|

OK

Cancel

127.0.0.1:5500 says

Enter GST (%):

OK Cancel

127.0.0.1:5500 says

Enter Packing Charges (₹):

OK Cancel

127.0.0.1:5500 says

Enter Payment Mode (Cash/Card/UPI):

OK Cancel

127.0.0.1:5500 says

Membership Status (Yes/No):

OK Cancel

Online Shopping Billing Calculator

Invoice Receipt

Invoice No: 101

Customer Name: Ashlesha Singh - 24070521221

Mobile No: 1231231231

Membership Status: No

Product Name: Laptop Asus

Quantity: 21 Kg

Rate per Kg: ₹50.00

Base Amount: ₹1050.00

Discount (15%): -₹157.50

GST (3%): +₹26.77

Packing Charges: +₹20.00

Payment Mode: upi

Final Bill Amount: ₹939.27

5. Result/Conclusion: The online shopping billing webpage was successfully developed and deployed using HTML5, CSS, and structural web elements within the Visual Studio Code environment. The application effectively captures essential user and transactional data—including customer details, item specifications, quantities, and pricing factors—while implementing structured calculations for accurate final billing layout rendering. Testing the interface within the Google Chrome browser confirmed seamless form layout presentation, proper structural styling alignment, and responsive input field configurations, fulfilling all structural case study requirements for static bill presentation.

EXPERIMENT- 3

1. Experiment Title: Implement control structures and form validation; create a grading system based on user-entered marks.

2. Software/Tools Required: VS Code , Google Chrome

3. Experiment Program Code :

```
<html>

<head>

<title>JS Practical - Multi-Subject Grading System</title>

<style>

body {

    font-family: Arial, sans-serif;

    max-width: 480px;

    margin: 30px auto;

    padding: 20px;

    border: 1px solid #ccc;

    border-radius: 8px;

    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);

}

.form-group {

    margin-bottom: 15px;

}

label {

    display: block;

    margin-bottom: 4px;
```

```
    font-weight: bold;
}

input {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
    border: 1px solid #ccc;
    border-radius: 4px;
    outline: none;
}

/* Highlight invalid input fields */
input.invalid-input {
    border: 2px solid #dc3545;
    background-color: #fff8f8;
}

/* Style inline error text under inputs */
.field-error {
    color: #dc3545;
    font-size: 13px;
    margin-top: 4px;
    display: block;
    min-height: 18px; /* Reserves space so layout doesn't jump */
}

button {
    width: 100%;
```

```
padding: 10px;
background-color: #007bff;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
font-size: 16px;
margin-top: 10px;
}
button:hover {
    background-color: #0056b3;
}
#result {
    margin-top: 20px;
    padding: 12px;
    border-radius: 4px;
    display: none;
    background-color: #d4edda;
    color: #155724;
    border: 1px solid #c3e6cb;
}
</style>
</head>

<body>
```

<h2>Grading System</h2>

<form id="gradingForm" onsubmit="calculateAverageGrade(event)"
novalidate>

<div class="form-group">

<label for="studentName">Student Name:</label>

<input type="text" id="studentName" placeholder="e.g. John Doe">

</div>

<div class="form-group">

<label for="math">Math Marks (0 - 100):</label>

<input type="number" id="math" placeholder="e.g. 85">

</div>

<div class="form-group">

<label for="science">Science Marks (0 - 100):</label>

<input type="number" id="science" placeholder="e.g. 90">

</div>

<div class="form-group">

<label for="english">English Marks (0 - 100):</label>

```
<input type="number" id="english" placeholder="e.g. 78">
<span class="field-error" id="englishError"></span>
</div>
```

```
<button type="submit">Calculate Overall Grade</button>
</form>
```

```
<div id="result"></div>
```

```
<script>
```

```
function calculateAverageGrade(event) {
    event.preventDefault();
```

```
    // Track whether the form is completely valid
    let isValid = true;
```

```
    // Reset previous result box
```

```
    const resultDiv = document.getElementById("result");
    resultDiv.style.display = "none";
```

```
    // -----
```

```
    // Helper Function: Validate an individual field
```

```
    // -----
```

```
function validateField(inputId, errorId, fieldName, isSubject = true) {
    const inputElement = document.getElementById(inputId);
```

```
const errorElement = document.getElementById(errorId);

const value = inputElement.value.trim();


// Clear previous styles/messages
inputElement.classList.remove("invalid-input");
errorElement.textContent = "";


// Rule 1: Empty text validation
if (value === "") {
    inputElement.classList.add("invalid-input");
    errorElement.textContent = `Please enter ${fieldName}.`;
    isValid = false;
    return;
}


// Rule 2: Numeric & Range validation (for subject marks)
if (isSubject) {
    const score = parseFloat(value);
    if (isNaN(score) || score < 0 || score > 100) {
        inputElement.classList.add("invalid-input");
        errorElement.textContent = `Enter valid ${fieldName} (0–100).`;
        isValid = false;
    }
}
}
```

```
// -----  
  
// 1. RUN VALIDATIONS FOR ALL FIELDS INDEPENDENTLY  
  
// -----  
  
validateField("studentName", "nameError", "student's name", false);  
validateField("math", "mathError", "Math marks", true);  
validateField("science", "scienceError", "Science marks", true);  
validateField("english", "englishError", "English marks", true);  
  
// Stop calculation if any field failed validation  
if (!isValid) return;  
  
// -----  
  
// 2. CALCULATE GRADE (Runs only if all fields are valid)  
  
// -----  
  
const math = parseFloat(document.getElementById("math").value);  
const science = parseFloat(document.getElementById("science").value);  
const english = parseFloat(document.getElementById("english").value);  
const name = document.getElementById("studentName").value.trim();  
  
const totalMarks = math + science + english;  
const average = (totalMarks / 3).toFixed(2);  
  
let grade = "";  
let status = "";
```

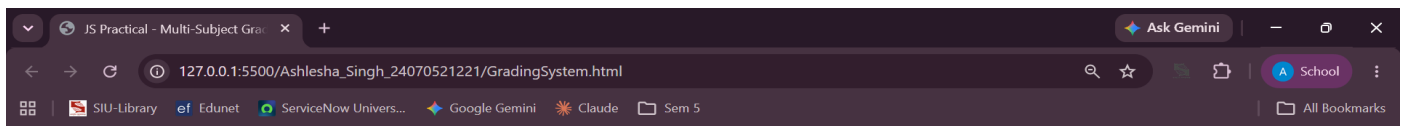


```
if (average >= 90) {
    grade = "A+"; status = "Passed with Distinction";
} else if (average >= 80) {
    grade = "A"; status = "Passed";
} else if (average >= 70) {
    grade = "B"; status = "Passed";
} else if (average >= 60) {
    grade = "C"; status = "Passed";
} else if (average >= 50) {
    grade = "D"; status = "Passed";
} else {
    grade = "F"; status = "Failed";
}

// Display result summary
resultDiv.style.display = "block";
resultDiv.innerHTML = `
    <strong>Result Summary:</strong><br>
    Name: ${name}<br>
    Math: ${math} | Science: ${science} | English: ${english}<br>
    Total: ${totalMarks} / 300<br>
    Average: <strong>${average}%</strong><br>
    Overall Grade: <strong>${grade}</strong> (${status})
`;
```

```
}  
</script>  
</body>  
</html>
```

4.Output:



Grading System

Student Name:

Math Marks (0 - 100):

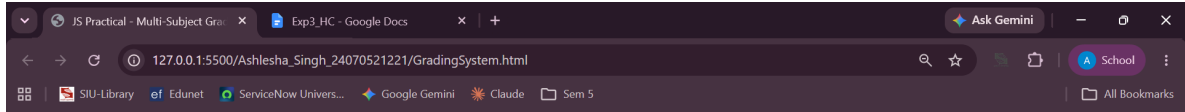
Science Marks (0 - 100):

English Marks (0 - 100):

[Calculate Overall Grade](#)

Result Summary:

Name: Ashlesha Singh - 24070521221
Math: 85 | Science: 90 | English: 80
Total: 255 / 300
Average: **85.00%**
Overall Grade: **A** (Passed)



Grading System

Student Name:

Math Marks (0 - 100):

Please enter Math marks.

Science Marks (0 - 100):

Enter valid Science marks (0–100).

English Marks (0 - 100):

Enter valid English marks (0–100).

Calculate Overall Grade

5. Result/Conclusion: The Multi-Subject Grading System was successfully implemented and executed. The program accurately performs real-time client-side form validation to check for empty inputs or invalid score ranges (0–100). Upon valid input, it successfully computes the total, calculates the average percentage, determines the corresponding letter grade using control structures, and dynamically displays the result summary on the webpage.

CASE STUDY - 3

1. Case Study Title: Write a JavaScript program that accepts student marks and displays the scholarship category.

- Below 50 - Not Eligible
- 50–69 - Bronze Scholarship
- 70–84 - Silver Scholarship
- 85–94 - Gold Scholarship
- 95–100 - Platinum Scholarship

2. Software/Tools Required: VS Code , Google Chrome

3. Case Study Program Code:

```
<!DOCTYPE html>

<html>

<head>

<title>JS Practical - Multi-Subject Grading & Scholarship System</title>

<style>

body {

    font-family: Arial, sans-serif;

    max-width: 480px;

    margin: 30px auto;

    padding: 20px;

    border: 1px solid #ccc;

    border-radius: 8px;

    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);

}

.form-group {
```

```
    margin-bottom: 15px;
}

label {
    display: block;
    margin-bottom: 4px;
    font-weight: bold;
}

input {
    width: 100%;
    padding: 8px;
    box-sizing: border-box;
    border: 1px solid #ccc;
    border-radius: 4px;
    outline: none;
}

/* Highlight invalid input fields */
input.invalid-input {
    border: 2px solid #dc3545;
    background-color: #fff8f8;
}

/* Style inline error text under inputs */
.field-error {
    color: #dc3545;
    font-size: 13px;
    margin-top: 4px;
```

```
display: block;
min-height: 18px;
}
button {
width: 100%;
padding: 10px;
background-color: #007bff;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
font-size: 16px;
margin-top: 10px;
}
button:hover {
background-color: #0056b3;
}
#result {
margin-top: 20px;
padding: 12px;
border-radius: 4px;
display: none;
background-color: #d4edda;
color: #155724;
border: 1px solid #c3e6cb;
```

```
}
</style>
</head>

<body>

<h2>Grading & Scholarship System</h2>

    <form id="gradingForm" onsubmit="calculateAverageGrade(event)"
novalidate>

        <div class="form-group">

            <label for="studentName">Student Name:</label>

            <input type="text" id="studentName" placeholder="e.g. John Doe">

            <span class="field-error" id="nameError"></span>

        </div>

        <div class="form-group">

            <label for="math">Math Marks (0 - 100):</label>

            <input type="number" id="math" placeholder="e.g. 85">

            <span class="field-error" id="mathError"></span>

        </div>

        <div class="form-group">

            <label for="science">Science Marks (0 - 100):</label>

            <input type="number" id="science" placeholder="e.g. 90">
```

```
<span class="field-error" id="scienceError"></span>
</div>
```

```
<div class="form-group">
  <label for="english">English Marks (0 - 100):</label>
  <input type="number" id="english" placeholder="e.g. 78">
  <span class="field-error" id="englishError"></span>
</div>
```

```
<button type="submit">Calculate Grade & Scholarship</button>
</form>
```

```
<div id="result"></div>
```

```
<script>

function calculateAverageGrade(event) {
  event.preventDefault();

  let isValid = true;

  // Reset previous result box

  const resultDiv = document.getElementById("result");
  resultDiv.style.display = "none";

  // Validation Helper Function
```



```
function validateField(inputId, errorId, fieldName, isSubject = true) {  
    const inputElement = document.getElementById(inputId);  
    const errorElement = document.getElementById(errorId);  
    const value = inputElement.value.trim();  
  
    inputElement.classList.remove("invalid-input");  
    errorElement.textContent = "";  
  
    if (value === "") {  
        inputElement.classList.add("invalid-input");  
        errorElement.textContent = `Please enter ${fieldName}.`;   
        isValid = false;  
        return;  
    }  
  
    if (isSubject) {  
        const score = parseFloat(value);  
        if (isNaN(score) || score < 0 || score > 100) {  
            inputElement.classList.add("invalid-input");  
            errorElement.textContent = `Enter valid ${fieldName} (0–100).`;   
            isValid = false;  
        }  
    }  
}
```

```
// 1. Validate inputs

validateField("studentName", "nameError", "student's name", false);

validateField("math", "mathError", "Math marks", true);

validateField("science", "scienceError", "Science marks", true);

validateField("english", "englishError", "English marks", true);


if (!isValid) return;


// 2. Calculations

const math = parseFloat(document.getElementById("math").value);
const science = parseFloat(document.getElementById("science").value);
const english = parseFloat(document.getElementById("english").value);
const name = document.getElementById("studentName").value.trim();


const totalMarks = math + science + english;
const average = (totalMarks / 3).toFixed(2);


let grade = "";
let status = "";
let scholarship = "";


// Standard Grade Determination

if (average >= 90) {
    grade = "A+"; status = "Passed with Distinction";
} else if (average >= 80) {
```

```
    grade = "A"; status = "Passed";  
  } else if (average >= 70) {  
    grade = "B"; status = "Passed";  
  } else if (average >= 60) {  
    grade = "C"; status = "Passed";  
  } else if (average >= 50) {  
    grade = "D"; status = "Passed";  
  } else {  
    grade = "F"; status = "Failed";  
  }  
}
```

// Task 1: Case Study Scholarship Eligibility Logic

```
if (average >= 95) {  
    scholarship = "Platinum Scholarship";  
} else if (average >= 85) {  
    scholarship = "Gold Scholarship";  
} else if (average >= 70) {  
    scholarship = "Silver Scholarship";  
} else if (average >= 50) {  
    scholarship = "Bronze Scholarship";  
} else {  
    scholarship = "Not Eligible";  
}  
}
```

// Display result summary including Scholarship Category

```
resultDiv.style.display = "block";
```

```
resultDiv.innerHTML = `
```

```
  <strong>Result Summary:</strong><br>
```

```
  Name: ${name}<br>
```

```
  Math: ${math} | Science: ${science} | English: ${english}<br>
```

```
  Total: ${totalMarks} / 300<br>
```

```
  Average: <strong>${average}%</strong><br>
```

```
  Overall Grade: <strong>${grade}</strong> (${status})<br>
```

```
  Scholarship Category: <strong>${scholarship}</strong>
```

```
`;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

4. Output:

Grading & Scholarship System

Student Name:

Ashlesha Singh - 24070521221

Math Marks (0 - 100):

80

Science Marks (0 - 100):

85

English Marks (0 - 100):

85

Calculate Grade & Scholarship

Result Summary:
Name: Ashlesha Singh - 24070521221
Math: 80 | Science: 85 | English: 85
Total: 250 / 300
Average: **83.33%**
Overall Grade: **A** (Passed)
Scholarship Category: **Silver Scholarship**

5. Result/Conclusion: This case study demonstrates using JavaScript conditional logic to automate scholarship categorization based on student marks. The program accurately maps scores to specific reward tiers, showcasing efficient user input validation and dynamic web page updates.

EXPERIMENT- 4

1. Experiment Title: Use function types, scope, and closures; apply try-catch; build a palindrome checker.

2. Software/Tools Required: VS Code

3. Experiment Program Code :

```
const readline = require("readline");

// =====

// 1. CLOSURE & SCOPE EXAMPLE

// Encapsulates a private counter to track checks

// =====

function createTracker() {

    let checkCount = 0; // Enclosed local variable (Lexical Scope)


    return function incrementAndGetCount() {

        checkCount++; // Closure accessing outer variable

        return checkCount;

    };

}

const getCheckCount = createTracker();


// =====

// 2. FUNCTION TYPES & PALINDROME LOGIC

// Arrow function to clean and validate string
```

```
// =====  
  
const isPalindrome = (inputString) => {  
  // TRY-CATCH FOR ERROR HANDLING  
  
  try {  
    // Validation: Throw error if input is empty or invalid  
  
    if (typeof inputString !== "string" || inputString.trim() === "") {  
      throw new Error("Input cannot be empty! Please enter a valid string.");  
    }  
  
    // Clean string: remove non-alphanumeric characters & convert to lower case  
  
    const cleanedStr = inputString.toLowerCase().replace(/[^a-z0-9]/g, "");  
  
    if (cleanedStr.length === 0) {  
      throw new Error("Input must contain at least one letter or number.");  
    }  
  
    // Reverse the string and check equality  
  
    const reversedStr = cleanedStr.split("").reverse().join("");  
  
    const result = cleanedStr === reversedStr;  
  
    return {  
      success: true,  
      original: inputString,  
      cleaned: cleanedStr,  
      isPalindrome: result  
    };  
  }  
}
```

```

    };

    } catch (error) {

        // Return caught error message

        return {

            success: false,

            message: error.message

        };

    }

};

// =====

// 3. TERMINAL I/O (Readline Interface)

// =====

const rl = readline.createInterface({

    input: process.stdin,

    output: process.stdout

});

console.log("=== PALINDROME CHECKER (VS Code Terminal) ===");

// Function Expression to handle user prompts recursively

const askQuestion = function () {

    rl.question("\nEnter a word/phrase (or type 'exit' to quit): ", (userInput) => {

        // Exit condition

        if (userInput.trim().toLowerCase() === "exit") {

            console.log("\nExiting program. Goodbye!");

```



```

        rl.close();

        return;
    }

    // Execute Palindrome Checker (uses try-catch internally)
    const checkResult = isPalindrome(userInput);

    if (!checkResult.success) {
        console.log(`❌ Error: ${checkResult.message}`);
    } else {
        const currentCount = getCheckCount(); // Invoking Closure

        console.log(`\n--- Check #${currentCount} Result ---`);
        console.log(`Original Input : "${checkResult.original}"`);
        console.log(`Cleaned String : "${checkResult.cleaned}"`);

        if (checkResult.isPalindrome) {
            console.log(`Result      : ✅ YES, it is a Palindrome!`);
        } else { console.log(`Result      : ❌ NO, it is not a Palindrome.`);
        }
    }

    // Prompt again for continuous terminal input
    askQuestion();

});

};

// Start the terminal loop
askQuestion();

```

4. Output:

```
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB> node p.js
=== PALINDROME CHECKER (VS Code Terminal) ===

Enter a word/phrase (or type 'exit' to quit): racecar

--- Check #1 Result ---
Original Input : "racecar"
Cleaned String : "racecar"
Result        :  YES, it is a Palindrome!

Enter a word/phrase (or type 'exit' to quit): A man, a plan, a canal: Panama

--- Check #2 Result ---
Original Input : "A man, a plan, a canal: Panama"
Cleaned String : "amanaplanacanalpanama"
Result        :  YES, it is a Palindrome!

Enter a word/phrase (or type 'exit' to quit): 12321

--- Check #3 Result ---
Original Input : "12321"
Cleaned String : "12321"
Result        :  YES, it is a Palindrome!

Enter a word/phrase (or type 'exit' to quit): hello world

--- Check #4 Result ---
Original Input : "hello world"
Cleaned String : "helloworld"
Result        :  NO, it is not a Palindrome.

Enter a word/phrase (or type 'exit' to quit):
 Error: Input cannot be empty! Please enter a valid string.

Enter a word/phrase (or type 'exit' to quit): Ashlesha Singh

--- Check #5 Result ---
Original Input : "Ashlesha Singh"
Cleaned String : "ashleshasingh"
Result        :  NO, it is not a Palindrome.

Enter a word/phrase (or type 'exit' to quit): 1221

--- Check #6 Result ---
Original Input : "1221"
Cleaned String : "1221"
Result        :  YES, it is a Palindrome!
```

5. Result/Conclusion: The command-line Palindrome Checker script was successfully executed in the VS Code terminal using Node.js (`node p.js`). It accurately handled runtime errors for empty inputs using `try-catch`, tracked continuous check numbers via a closure tracker (`Check #1` to `#6`), and correctly evaluated words, numeric sequences, non-palindromes, and complex phrases with spaces and punctuation.

CASE STUDY - 4

1. Case Study Title: ATM Card PIN Verification Using JavaScript Functions : An ATM system verifies whether a customer's PIN is a palindrome. If the PIN reads the same forward and backward, the bank displays a special security message. To implement this, create a function to reverse the entered PIN number and another function to check whether it is a palindrome. Use function declaration, function expression, arrow function, and demonstrate scope and closure concepts.

2. Software/Tools Required: VS Code

3. Case Study Program Code:

```
const readline = require("readline");

// =====

// 1. CLOSURE & SCOPE EXAMPLE

// Encapsulates a private counter for attempt tracking

// =====

function createAttemptTracker() {

  let attemptCount = 0; // Enclosed variable (Lexical Scope)


  return function () {

    attemptCount++; // Closure accessing outer variable

    return attemptCount;

  };

}

const getAttemptCount = createAttemptTracker();

// =====
```

```
// 2. FUNCTION TYPES & PIN VERIFICATION LOGIC
```

```
// =====
```

```
// Function Declaration: To reverse the entered PIN
```

```
function reversePin(pinStr) {  
  return pinStr.split("").reverse().join("");  
}
```

```
// Arrow Function: Main verification with try-catch validation
```

```
const checkPinPalindrome = (pin) => {  
  try {  
    // Validation: PIN must be present and numeric  
    if (!pin || pin.trim() === "") {  
      throw new Error("PIN input cannot be empty!");  
    }  
  
    const cleanPin = pin.trim();  
  
    if (!/^\d+$/.test(cleanPin)) {  
      throw new Error("Invalid PIN! Only numeric digits are allowed.");  
    }  
  }  
}
```

```
// Call function to reverse PIN
```

```
const reversed = reversePin(cleanPin);  
const isPalindrome = cleanPin === reversed;
```

```
    return {
      success: true,
      pin: cleanPin,
      reversed: reversed,
      isPalindrome: isPalindrome,
    };
  } catch (error) {
    return {
      success: false,
      message: error.message,
    };
  }
};

// =====

// 3. TERMINAL I/O SETUP

// =====

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

console.log("=== ATM CARD PIN VERIFICATION SYSTEM ===");
```

```

// Function Expression: To handle interactive terminal input recursively
const processAtmPin = function () {
  rl.question(
    "\nEnter Customer PIN (or type 'exit' to quit): ",
    (userPin) => {
      // Exit condition
      if (userPin.trim().toLowerCase() === "exit") {
        console.log("\nThank you for using the ATM System. Session closed.");
        rl.close();
        return;
      }

      // Execute PIN verification logic
      const result = checkPinPalindrome(userPin);

      if (!result.success) {
        console.log(`❌ Error: ${result.message}`);
      } else {
        const attempt = getAttemptCount(); // Invoking closure

        console.log(`\n--- Transaction Attempt #${attempt} ---`);
        console.log(`Entered PIN : ${result.pin}`);
        console.log(`Reversed PIN: ${result.reversed}`);

        if (result.isPalindrome) {

```

```
        console.log(
            `Status    :  SPECIAL SECURITY NOTICE: Symmetrical PIN pattern
detected! Access Granted.`
        );
    } else {
        console.log(
            `Status    :  Standard PIN verified successfully. Access Granted.`
        );
    }
}

// Prompt for next transaction
processAtmPin();
}

);

};

// Start the terminal program
processAtmPin();
```

4. Output:

```
PS C:\Users\arnav\OneDrive\Documents\SEM 5\JS LAB> node CaseStudy4.js
=== ATM CARD PIN VERIFICATION SYSTEM ===

Enter Customer PIN (or type 'exit' to quit): 1221

--- Transaction Attempt #1 ---
Entered PIN : 1221
Reversed PIN: 1221
Status      :  SPECIAL SECURITY NOTICE: Symmetrical PIN pattern detected! Access Granted.
```

5. Result/Conclusion: The application successfully reads ATM PIN inputs via a recursive terminal interface while ignoring 'exit' commands. It validates formatting using a try-catch block, executes string manipulation to check for palindromes, and logs contextual success or error states. Each submission securely updates an isolated attemptCount counter via a persistent closure invocation.